PYTHON • OBJECT-ORIENTED PROGRAMMING

Python OOPS
Complete Interview & Placement Revision Notes

From Basics to Interview-Level Concepts | Examples | Output Prediction | Coding Practice

13 Core Concepts | 40+ Interview Questions | 15 Output Predictions | 15 Coding Problems | Last-Minute Checklist

---

Built for **B.Tech CSE students** • **Freshers preparing for placements** • **Python developers** • **Technical interview aspirants**

Save this • Revise this • Ace the OOPS round

Contents

Everything you need before a Python OOPS interview — organized for fast revision.

CORE CONCEPTS

01 OOPS Introduction

02 Class & Object

03 Constructor

04 Encapsulation

05 Inheritance

06 super()

07 Method Overriding

08 Polymorphism

09 Operator Overloading

10 Duck Typing

11 Abstraction

12 Important OOPS Concepts

13 Dunder / Magic Methods

REVISION & PRACTICE

14 Interview Traps

15 30-Second Revision

16 Output Prediction (15 Snippets)

17 Interview Questions (40+)

18 Coding Practice (15 Problems)

19 Last-Minute Interview Checklist

SECTION 01

OOPS Introduction

The foundation every Python interview starts with

1.1 What is OOPS?

DEFINITION

Object-Oriented Programming (OOP) is a programming style that organizes code around **objects** — self-contained units that bundle **data (attributes)** and **behavior (methods)** together, instead of writing logic as a sequence of standalone functions acting on separate data.

WHY IT IS USED

Real software models real entities — a Student, a BankAccount, a Car. OOP lets code mirror these entities directly, which makes large codebases easier to design, extend, test, and reuse.

REAL-WORLD ANALOGY

Think of a **car**. It has data (color, speed, fuel level) and behavior (start, brake, accelerate). You don't need to know how the engine works internally to drive it — you just use the steering wheel and pedals. That's exactly how a Python object works: internal data + exposed behavior.

1.2 Why OOPS? (Procedural vs Object-Oriented)

DEFINITION

In procedural programming, data and functions are separate — functions are passed data to operate on. In OOP, data and the functions that operate on it live together inside one unit (the object), which reduces accidental misuse of data and duplication of logic.

Aspect	Procedural	Object-Oriented
Focus	Functions / steps	Objects that hold data + behavior
Data safety	Data is often global / exposed	Data can be hidden (encapsulation)
Reusability	Copy-paste functions	Inheritance reuses code
Real-world mapping	Weak	Strong (Student, Car, Employee...)
Scalability	Harder for large systems	Easier to scale and maintain

1.3 The Four Pillars of OOPS



Encapsulation — bundling data and methods, restricting direct access to internal state.
Inheritance — a class acquiring properties and behavior from another class.
Polymorphism — the same interface/method behaving differently for different objects.
Abstraction — hiding implementation detail and exposing only what's necessary.

INTERVIEW POINT

Interviewers frequently ask "name and explain the four pillars in one line each" — practice saying all four in under 30 seconds without pausing.

SECTION 02

Class & Object

The building blocks of every OOP program

2.1 Class Definition

DEFINITION

A class is a blueprint/template that defines what attributes and methods its objects will have. No memory for attribute values is allocated until an object is created.

WHY IT IS USED

Classes let you define a structure once and create many objects from it, instead of repeating variables and functions for every entity.

SYNTAX

```
class ClassName:
    # attributes
    # methods
```

EXAMPLE

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def display(self):
        print(self.name, self.marks)
```

LINE-BY-LINE

class Student: declares the blueprint. **__init__** runs automatically when an object is created and sets up instance data. **self** refers to the specific object being built.

> **REAL-WORLD ANALOGY**
> A class is like a *cookie cutter* — it defines the shape, but doesn't produce the cookie by itself.

> **COMMON BEGINNER MISTAKE**
> Writing logic directly under the class body expecting it to run per-object — code inside a class (outside methods) runs only once, at class-definition time.

> **INTERVIEW POINT**
> A class itself does not occupy memory for instance attributes — memory is allocated only when `__init__` runs on object creation.

2.2 Object Creation

DEFINITION

An object (also called an instance) is a concrete realization of a class — it has its own copy of instance attributes and can call the class's methods.

WHY IT IS USED

Objects are what your program actually works with at runtime; the class is only the design.

SYNTAX

```
object_name = ClassName(arguments)
```

EXAMPLE

```
s1 = Student("Riya", 91)
s2 = Student("Aman", 78)
s1.display()
s2.display()
```

OUTPUT

```
Riya 91
Aman 78
```

LINE-BY-LINE

Two independent objects `s1` and `s2` are created from the same class, each with its own `name` and `marks`.

REAL-WORLD ANALOGY

One class (blueprint) → many objects (houses), each with its own furniture (attribute values) but the same floor plan (structure).

COMMON BEGINNER MISTAKE

Confusing the class with an object — you cannot call an instance method directly on the class name without an instance, e.g. `Student.display()` fails without passing an object.

INTERVIEW POINT

Every object has its own independent copy of instance attributes, but shares the same method code defined in the class.

2.3 Attributes & Methods, Multiple Objects

DEFINITION

Attributes are variables bound to an object (or class); methods are functions defined inside a class that operate on that data, always receiving the object itself as the first parameter (`self`).

WHY IT IS USED

Grouping attributes and methods together keeps related data and logic in one place, which is the core idea of encapsulation.

EXAMPLE

```
class Car:
    wheels = 4  # class attribute
    def __init__(self, brand):
        self.brand = brand  # instance attribute

    def honk(self):
        print(f"{self.brand} says Beep!")

c1 = Car("Tata")
c2 = Car("Kia")
c1.honk(); c2.honk()
print(c1.wheels, c2.wheels)
```

OUTPUT

```
Tata says Beep!
Kia says Beep!
4 4
```

LINE-BY-LINE

`wheels` is shared by all Car objects (class attribute). `brand` is unique per object (instance attribute). Both objects share the same `wheels` value unless overridden on an instance.

REAL-WORLD ANALOGY

A real-world Student example: every student (object) has their own `roll_number` and `marks` (instance attributes), but all belong to the same `school_name` (class attribute) unless changed individually.

COMMON BEGINNER MISTAKE

Modifying a mutable class attribute (like a list) through one object accidentally changes it for all objects, because it's shared, not copied.

INTERVIEW POINT

Class attributes are stored once at the class level; instance attributes are stored per-object in the object's `__dict__`.

SECTION 03

Constructor

Setting up an object the moment it is born

3.1 __init__() — The Constructor

DEFINITION

`__init__()` is a special method Python calls automatically right after an object is created, used to initialize instance attributes. It is not technically the 'true' constructor (that's `__new__`), but for interviews and daily use it is treated as the constructor.

WHY IT IS USED

Without it, you'd have to manually set every attribute after creating each object — repetitive and error-prone.

SYNTAX

```
def __init__(self, params):
    self.attribute = params
```

EXAMPLE

```
class Book:
    def __init__(self, title):
        self.title = title
        print("Book created!")

b = Book("Python 101")
```

OUTPUT

```
Book created!
```

LINE-BY-LINE

As soon as `Book("Python 101")` runs, Python creates the object and immediately calls `__init__` on it, printing the message.

> **REAL-WORLD ANALOGY**
> Like a new employee's *onboarding form* — filled out automatically the moment they join, before they do any actual work.

> **COMMON BEGINNER MISTAKE**
> Thinking `__init__` creates the object. It doesn't — the object already exists by the time `__init__` runs; `__init__` only initializes it.

> **INTERVIEW POINT**
> `__init__` never returns a value other than `None` — returning anything else raises a `TypeError`.

3.2 self — The Instance Reference

DEFINITION

`self` is the first parameter of every instance method; it refers to the specific object the method is being called on, letting the method read/modify that object's own attributes.

WHY IT IS USED

Without `self`, a method would have no way to know which object's data to work with when many objects exist.

SYNTAX

```
def method(self, ...):
    self.attribute
```

EXAMPLE

```
class Counter:
    def __init__(self):
        self.count = 0
    def increase(self):
        self.count += 1

a = Counter(); b = Counter()
a.increase(); a.increase(); b.increase()
print(a.count, b.count)
```

OUTPUT

```
2 1
```

LINE-BY-LINE

a and b are separate objects; `self` inside `increase()` refers to whichever object called it, so their counts stay independent.

REAL-WORLD ANALOGY

`self` is like saying 'my' — 'my count', 'my name' — every object uses the same method code but applies it to 'its own' data.

COMMON BEGINNER MISTAKE

Forgetting to include `self` as the first parameter in an instance method definition causes a `TypeError` about missing/extra arguments when the method is called.

INTERVIEW POINT

`self` is just a naming convention, not a keyword — you could technically name it anything, but never do this in real code.

3.3 Parameterized vs Default Constructor

DEFINITION

A **parameterized constructor** accepts arguments to initialize attributes with custom values. Python has no separate 'default constructor' syntax like Java/C++ — if you don't define `__init__` at all, Python silently uses an implicit no-argument one that does nothing.

WHY IT IS USED

Parameterized constructors let each object start with different, meaningful data right from creation.

SYNTAX

```
def __init__(self, a, b=10):
    ...
```

EXAMPLE

```
class Point:
    def __init__(self, x=0, y=0):
        self.x = x
        self.y = y

p1 = Point()
p2 = Point(3, 4)
print(p1.x, p1.y)
print(p2.x, p2.y)
```

OUTPUT

```
0 0
3 4
```

LINE-BY-LINE

`p1` uses the default values `(0, 0)` since no arguments were passed; `p2` overrides them with `(3, 4)`.

REAL-WORLD ANALOGY

Like a pizza order form with default toppings pre-checked — you can accept the defaults or specify your own.

COMMON BEGINNER MISTAKE

Assuming Python has a distinct 'default constructor' concept like Java — in Python it's simply `__init__` with default parameter values, or no `__init__` at all.

INTERVIEW POINT

If a class defines no `__init__`, Python still lets you create objects — `class A: pass; A()` works fine.

SECTION 04

Encapsulation

Bundling data with behavior and controlling access

4.1 Public, Protected & Private Members

DEFINITION

Encapsulation restricts direct access to an object's internal data. Python has no true 'private' enforcement — it uses naming conventions: **public** (normal name), **protected** (single underscore `_var`, 'internal use' convention), and **private** (double underscore `__var`, triggers name mangling).

WHY IT IS USED

It protects internal state from being changed in invalid ways from outside the class, and hides implementation detail the caller doesn't need.

SYNTAX

```
self.public = x
self._protected = x
self.__private = x
```

EXAMPLE

```
class Account:
    def __init__(self, balance):
        self.owner = "Riya"       # public
        self._bank_code = "SBI"    # protected
        self.__balance = balance   # private

a = Account(1000)
print(a.owner)
print(a._bank_code)
print(a._Account__balance)  # name-mangled access
```

OUTPUT

```
Riya
SBI
1000
```

LINE-BY-LINE

`owner` is freely accessible. `_bank_code` is accessible too — the underscore is only a convention, not enforced. `__balance` is renamed internally to `_Account__balance` (name mangling), so `a.__balance` would fail, but the mangled name still works.

REAL-WORLD ANALOGY

Like an ATM: your balance (private) is never shown to a stranger, but the bank's branch code (protected) is visible on your card for context, and your name (public) is right on the card too.

COMMON BEGINNER MISTAKE

Believing `__balance` makes data truly private/unhackable in Python — it only renames the attribute; it is still technically reachable via `_ClassName__attr`.

INTERVIEW POINT
Say clearly in interviews: 'Python relies on convention and name mangling, not real access control, for private members.'

4.2 Getters & Setters (Traditional Style)

DEFINITION
Getter and setter methods provide controlled read/write access to a private attribute, letting you validate or transform values instead of allowing raw direct assignment.

WHY IT IS USED
They let you add validation logic (e.g., 'balance cannot be negative') without exposing the raw attribute.

SYNTAX

```
def get_x(self): return self.__x
def set_x(self, val): self.__x = val
```

EXAMPLE

```
class Account:
    def __init__(self, balance):
        self.__balance = balance
    def get_balance(self):
        return self.__balance
    def set_balance(self, amount):
        if amount < 0:
            print("Invalid amount")
        else:
            self.__balance = amount

a = Account(500)
a.set_balance(-100)
print(a.get_balance())
```

OUTPUT

```
Invalid amount
500
```

LINE-BY-LINE
`set_balance` rejects the negative amount, so `__balance` stays unchanged at 500, proving the validation worked.

REAL-WORLD ANALOGY
Like a bank teller (setter) who checks your withdrawal slip before touching the vault (private data), instead of letting you into the vault directly.

COMMON BEGINNER MISTAKE
Writing getters/setters for every single attribute 'because Java does it' — in Python, if no validation is needed, a plain public attribute is perfectly fine and more idiomatic.

INTERVIEW POINT
Modern Python typically prefers the `@property` decorator (covered in Section 12) over manual `get_/set_` methods.

SECTION 05

Inheritance

Reusing and extending existing classes

5.1 What is Inheritance? (IS-A relationship)

DEFINITION

Inheritance lets a class (child/derived) acquire the attributes and methods of another class (parent/base), modeling an **IS-A** relationship — e.g. a Dog IS-A Animal.

WHY IT IS USED

It avoids rewriting common logic in every related class — write it once in the parent, reuse it in every child.

```
class Animal:
    def eat(self):
        print("This animal eats food")

class Dog(Animal):
    def bark(self):
        print("Dog barks")

d = Dog()
d.eat()
d.bark()
```

OUTPUT

```
This animal eats food
Dog barks
```

REAL-WORLD ANALOGY

A child inherits traits (eye color, height tendency) from a parent, plus develops traits of their own — same idea as a Dog inheriting Animal's behavior while adding `bark()`.

Type	Structure	Example
Single	One parent → one child	Animal → Dog
Multilevel	Grandparent → parent → child chain	Animal → Dog → Puppy
Hierarchical	One parent → many children	Animal → Dog, Cat, Cow
Multiple	Child inherits from 2+ parents at once	class Child(Father, Mother)
Hybrid	Combination of the above types	Mix of hierarchical + multiple

5.2 Multilevel & Hierarchical Inheritance

DEFINITION

Multilevel inheritance chains classes (A → B → C), where C indirectly inherits from A too. Hierarchical inheritance has one parent with multiple independent children.

WHY IT IS USED

Multilevel models progressively specialized entities (Animal → Mammal → Dog); hierarchical models multiple variants sharing one base (Dog, Cat, Cow all being Animals).

EXAMPLE

```
class Animal:
    def eat(self): print("eats")
class Mammal(Animal):
    def walk(self): print("walks")
class Dog(Mammal):
    def bark(self): print("barks")

d = Dog()
d.eat(); d.walk(); d.bark()
```

OUTPUT

```
eats
walks
barks
```

LINE-BY-LINE

Dog gets `walk()` directly from `Mammal` and `eat()` indirectly from `Animal`, two levels up.

> **REAL-WORLD ANALOGY**
> A family tree: grandparent → parent → child. Traits can pass down more than one generation.

> **COMMON BEGINNER MISTAKE**
> Making inheritance chains too deep (5-6 levels) — it becomes hard to trace where a method actually comes from. Prefer composition when the chain gets long.

> **INTERVIEW POINT**
> Interviewers may ask you to draw the class hierarchy and identify which methods are inherited at which level.

5.3 Multiple & Hybrid Inheritance + MRO

DEFINITION

Multiple inheritance lets a class inherit from more than one parent directly. Python resolves method lookup order using the **MRO (Method Resolution Order)**, based on the C3 linearization algorithm, viewable via `ClassName.__mro__`.

WHY IT IS USED

Multiple inheritance is used to combine independent capabilities from unrelated classes (e.g. a class that is both 'Flyable' and 'Swimmable').

EXAMPLE

```
class Father:
    def skills(self): print("Gardening")
class Mother:
    def skills(self): print("Painting")
class Child(Father, Mother):
    pass

c = Child()
c.skills()
print(Child.__mro__[0:2])
```

OUTPUT

```
Gardening
(<class '__main__.Child'>, <class '__main__.Father'>)
```

LINE-BY-LINE

Since `Father` is listed first in `class Child(Father, Mother)`, Python's MRO checks `Father` before `Mother`, so `Father`'s `skills()` wins.

> **REAL-WORLD ANALOGY**
> Like a child picking up hobbies from both parents — if both parents teach the same hobby differently, the child usually follows whichever parent is asked first.

> **COMMON BEGINNER MISTAKE**
> Assuming Python behaves like C++ with ambiguous multiple-inheritance errors — Python always resolves it deterministically via MRO, never throws an ambiguity error by itself.

> **INTERVIEW POINT**
> 'Diamond problem' (A → B, C → D where B, C both inherit A) is elegantly solved by Python's C3-linearized MRO, unlike raw C++.

SECTION 06

super()

Calling the parent class from a child

6.1 Why super() is Used

DEFINITION

`super()` returns a proxy object that lets a child class call methods (including `__init__`) of its parent class, without hardcoding the parent's class name.

WHY IT IS USED

It lets the child extend the parent's behavior instead of completely rewriting it, and keeps code working correctly even if the parent class name changes or inheritance chain changes.

SYNTAX

```
super().__init__(args)
super().method_name(args)
```

EXAMPLE

```
class Animal:
    def __init__(self, name):
        self.name = name

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)
        self.breed = breed

d = Dog("Rex", "Labrador")
print(d.name, d.breed)
```

OUTPUT

```
Rex Labrador
```

LINE-BY-LINE

`Dog.__init__` calls `super().__init__(name)`, which runs `Animal`'s constructor to set `self.name`, then `Dog` adds its own `breed` attribute.

REAL-WORLD ANALOGY

Like a new employee following the company's general onboarding process (parent) first, then going through a role-specific extra briefing (child) on top of it.

COMMON BEGINNER MISTAKE

Forgetting to call `super().__init__()` in the child's constructor when the parent also needs initialization — the parent's attributes simply never get set.

INTERVIEW POINT

`super()` uses the MRO, not a hardcoded reference to the immediate parent — this matters heavily in multiple inheritance.

6.2 super() with Method Overriding

DEFINITION

When a child overrides a parent method but still wants the parent's version of the logic to run too, it calls `super().method()` inside its own overriding method.

WHY IT IS USED

This avoids duplicating the parent's logic when the child only needs to add something extra to it.

EXAMPLE

```
class Animal:
    def speak(self):
        print("Animal makes a sound")

class Dog(Animal):
    def speak(self):
        super().speak()
        print("Dog barks")

Dog().speak()
```

OUTPUT

```
Animal makes a sound
Dog barks
```

LINE-BY-LINE

`Dog.speak()` overrides `Animal.speak()` but calls `super().speak()` first to still run the parent's version before adding its own line.

REAL-WORLD ANALOGY

Like extending a recipe: 'follow the base sauce recipe (super), then add extra spice (child's own code)' instead of rewriting the sauce from scratch.

COMMON BEGINNER MISTAKE

Calling `Animal.speak(self)` directly instead of `super().speak()` — it works for single inheritance but breaks the MRO chain in multiple inheritance.

INTERVIEW POINT

'When would you NOT call super() in an override?' — when you intentionally want to fully replace the parent's behavior.

SECTION 07

Method Overriding

Redefining parent behavior in a child class

7.1 Method Overriding

DEFINITION

Method overriding happens when a child class defines a method with the exact same name (and signature) as one in its parent, replacing the parent's behavior when called on a child object.

WHY IT IS USED

It lets each subclass provide its own specific behavior for a shared method name defined generically in the parent.

SYNTAX

```
class Child(Parent):
    def method(self):
        # new behavior
```

EXAMPLE

```
class Shape:
    def area(self):
        print("Area not defined")

class Circle(Shape):
    def area(self):
        print("Area = pi * r^2")

s = Shape()
c = Circle()
s.area()
c.area()
```

OUTPUT

```
Area not defined
Area = pi * r^2
```

LINE-BY-LINE

Both classes define `area()`. Python decides which version runs based on the object's actual type at runtime — this is what enables runtime polymorphism.

REAL-WORLD ANALOGY

A generic 'Manager' job description gets overridden by a specific 'Sales Manager' job description that redefines certain duties while keeping the same title/role.

COMMON BEGINNER MISTAKE

Confusing overriding (same class hierarchy, same signature, runtime resolved) with overloading (same class, multiple signatures — not natively supported in Python).

INTERVIEW POINT

Overriding is the actual mechanism behind runtime polymorphism in Python — always connect these two topics in your answer.

Aspect	Method Overriding
Where	Between parent and child class
Signature	Same method name & parameters
Resolved	At runtime, based on object's actual class
Purpose	Provide subclass-specific behavior

SECTION 08

Polymorphism

One interface, many behaviors

8.1 What is Polymorphism?

DEFINITION

Polymorphism ('many forms') means the same method name or operator behaves differently depending on the object or data type it is used with.

WHY IT IS USED

It lets you write generic code (like a loop calling `.speak()` on any animal) without checking each object's exact type.

EXAMPLE

```
class Cat:
    def speak(self): print("Meow")
class Cow:
    def speak(self): print("Moo")

for animal in [Cat(), Cow()]:
    animal.speak()
```

OUTPUT

```
Meow
Moo
```

LINE-BY-LINE

The same call `animal.speak()` produces different output depending on which object is currently being iterated — that's polymorphism in action.

REAL-WORLD ANALOGY

A universal remote's 'power' button — pressing it on a TV, an AC, or a speaker triggers a different device-specific action, but the button/interface is identical.

INTERVIEW POINT

Give this one-liner in interviews: 'Polymorphism means same interface, different implementation, decided at runtime.'

8.2 Method Overloading — Python's Reality Check

IMPORTANT CLARIFICATION

Python does NOT support traditional method overloading the way Java or C++ does. In Java/C++ you can define multiple methods with the same name but different parameter lists, and the compiler picks the right one. In Python, defining a method twice simply makes the second definition **overwrite** the first — there is no signature-based selection.

```
class Demo:
    def show(self, a):
        print(a)
    def show(self, a, b):   # this overwrites the first show()
        print(a, b)

d = Demo()
d.show(1, 2)
# d.show(1)  -> TypeError: missing argument b
```

OUTPUT

```
1 2
```

PYTHON'S ALTERNATIVES TO OVERLOADING

Since true overloading isn't available, Python achieves similar flexibility using:

```
# 1. Default arguments
def greet(name, msg="Hello"):
    print(msg, name)

# 2. *args / **kwargs (variable arguments)
def add(*nums):
    return sum(nums)

greet("Riya")            # Hello Riya
greet("Aman", "Hi")      # Hi Aman
print(add(1, 2))         # 3
print(add(1, 2, 3, 4))   # 10
```

INTERVIEW POINT

This is one of the most commonly asked OOPS questions — always explicitly say: "Python doesn't support compile-time method overloading; we simulate it using default arguments or *args/**kwargs."

Aspect	Method Overloading	Method Overriding
Classes involved	Same class	Parent & child class
Native support in Python?	No (simulated via defaults/*args)	Yes, natively
When resolved	N/A in Python (no true overloading)	Runtime, based on object type
Purpose	Same operation, different input forms	Subclass-specific behavior

SECTION 09

Operator Overloading

Making built-in operators work with your own objects

9.1 Operator Overloading via Dunder Methods

DEFINITION

Operator overloading lets you redefine how a built-in operator (+, -, *, ==...) behaves when applied to objects of your own class, by implementing the corresponding dunder method.

WHY IT IS USED

Without it, operators like + have no idea how to combine two custom objects (e.g. two Point objects); overloading gives your objects natural, readable syntax.

SYNTAX

```
def __add__(self, other): ...
def __sub__(self, other): ...
def __mul__(self, other): ...
def __eq__(self, other): ...
```

EXAMPLE

```
class Point:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Point(self.x + other.x, self.y + other.y)
    def __eq__(self, other):
        return self.x == other.x and self.y == other.y
    def __str__(self):
        return f"({self.x}, {self.y})"

p1 = Point(1, 2)
p2 = Point(3, 4)
print(p1 + p2)
print(p1 == Point(1, 2))
```

OUTPUT

```
(4, 6)
True
```

LINE-BY-LINE

`p1 + p2` internally calls `p1.__add__(p2)`, returning a new Point. `p1 == Point(1, 2)` calls `__eq__` to compare coordinates instead of comparing object identity.

> **REAL-WORLD ANALOGY**
> Like teaching a translator (your class) what '+' means in a new language — the symbol stays the same, but you define what action it triggers for your specific objects.

COMMON BEGINNER MISTAKE

Forgetting that without `__eq__`, == on custom objects compares identity (memory address) by default, not attribute values — leading to unexpected `False` results.

INTERVIEW POINT

Know the full mapping: + → `__add__`, - → `__sub__`, * → `__mul__`, == → `__eq__`, < → `__lt__`.

Operator	Dunder Method	Typical Meaning
+	__add__	Combine / sum two objects
-	__sub__	Subtract one object from another
*	__mul__	Scale / multiply
==	__eq__	Value-based equality check
<	__lt__	Less-than comparison (used by sort())

SECTION 10

Duck Typing

If it walks like a duck and quacks like a duck...

10. Duck Typing

DEFINITION

Duck typing means Python cares about whether an object **has the method/attribute you're calling**, not what class it officially belongs to. Type checking happens implicitly at the point of use, not through explicit type declarations.

WHY IT IS USED

It gives Python flexibility — functions can accept any object that 'behaves' correctly, regardless of inheritance, enabling flexible, reusable code.

EXAMPLE

```
class Duck:
    def sound(self): print("Quack")
class Person:
    def sound(self): print("I am imitating a duck")

def make_it_sound(entity):
    entity.sound()

make_it_sound(Duck())
make_it_sound(Person())
```

OUTPUT

```
Quack
I am imitating a duck
```

LINE-BY-LINE

`make_it_sound()` never checks the type of `entity` — it just calls `.sound()`. Both `Duck` and unrelated `Person` work, because both define `sound()`.

REAL-WORLD ANALOGY

'If it walks like a duck and quacks like a duck, treat it as a duck' — a referee doesn't check a player's ID card mid-game, only whether they can perform the action expected of them.

COMMON BEGINNER MISTAKE

Assuming duck typing requires inheritance from a common base class — it explicitly does NOT; the two classes above share no relationship at all.

INTERVIEW POINT

Duck typing is why Python is called a **dynamically typed** language, and it underlies how many built-in functions (like `len()`) work on very different object types.

SECTION 11

Abstraction

Hiding the how, exposing only the what

11. Why Abstraction? ABC & abstractmethod

DEFINITION

Abstraction means exposing only the essential features of an object while hiding internal implementation detail. In Python it is formally implemented using the `abc` module's `ABC` base class and the `@abstractmethod` decorator.

WHY IT IS USED

It forces subclasses to implement certain required methods, and lets you define a common interface/contract without dictating how each subclass fulfills it.

SYNTAX

```
from abc import ABC, abstractmethod
class Base(ABC):
    @abstractmethod
    def method(self): pass
```

EXAMPLE

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass

class Rectangle(Shape):
    def __init__(self, l, w):
        self.l, self.w = l, w
    def area(self):
        return self.l * self.w

r = Rectangle(4, 5)
print(r.area())
# Shape()  -> TypeError: Can't instantiate abstract class
```

OUTPUT

```
20
```

LINE-BY-LINE

Shape is an abstract class — it cannot be instantiated directly because it has an unimplemented `@abstractmethod`. `Rectangle` is a concrete class because it implements `area()`, so it can be instantiated.

REAL-WORLD ANALOGY

Like a car's steering wheel — the driver (caller) only needs to know 'turn it to steer'; the internal steering mechanism (implementation) is completely hidden and can differ between car models.

COMMON BEGINNER MISTAKE

Trying to instantiate an abstract class directly (`Shape()`) — Python raises `TypeError: Can't instantiate abstract class Shape with abstract method area`.

INTERVIEW POINT

Distinguish clearly: **Encapsulation** hides *data* (via access control); **Abstraction** hides *implementation complexity* (via abstract interfaces). They're related but not the same.

Term	Abstract Class	Concrete Class
Can be instantiated?	No	Yes
Contains abstractmethod?	At least one	None left unimplemented
Purpose	Defines a contract/interface	Provides real, usable behavior

SECTION 12

Important Python OOPS Concepts

The details that separate a fresher answer from an interview-ready answer

12. Instance Variables vs Class Variables

DEFINITION

Instance variables are declared inside methods (usually `__init__`) using `self.` and are unique to each object. Class variables are declared directly inside the class body and are shared by all objects unless explicitly overridden on an instance.

WHY IT IS USED

Class variables are ideal for data that should be identical/shared across every object (e.g. a company name); instance variables hold data that naturally differs per object.

EXAMPLE

```
class Employee:
    company = "TechCorp"       # class variable
    def __init__(self, name):
        self.name = name        # instance variable

e1 = Employee("Riya")
e2 = Employee("Aman")
Employee.company = "NewCorp"
print(e1.company, e2.company)
```

OUTPUT

```
NewCorp NewCorp
```

LINE-BY-LINE

Both objects see the updated `company` because it's stored once on the class, not copied per object; changing it on the class affects every instance that hasn't overridden it locally.

REAL-WORLD ANALOGY

A class variable is like a company-wide holiday calendar — one copy, applies to every employee. An instance variable is like an employee's personal ID number — unique to each person.

COMMON BEGINNER MISTAKE

Doing `e1.company = "X"` creates a brand-new instance variable named `company` on e1 only — it does NOT change the shared class variable, and can silently 'shadow' it.

INTERVIEW POINT

'What happens if you assign to a class variable through an instance?' → It creates a new instance attribute shadowing the class one; the class variable itself is untouched.

12. Instance, Class & Static Methods (self vs cls)

DEFINITION

Instance methods take `self` and access object data. **Class methods** (`@classmethod`) take `cls` and operate on the class itself, often used as alternate constructors. **Static methods** (`@staticmethod`) take neither `self` nor `cls` — they're just utility functions grouped inside the class namespace.

WHY IT IS USED

This distinction lets you write logic that correctly targets the object, the class, or neither, instead of forcing everything through `self`.

SYNTAX

```
@classmethod
def method(cls, ...): ...
@staticmethod
def method(...): ...
```

EXAMPLE

```
class Employee:
    count = 0
    def __init__(self, name):
        self.name = name
        Employee.count += 1

    @classmethod
    def total(cls):
        return cls.count

    @staticmethod
    def is_valid_name(name):
        return len(name) > 0

e1 = Employee("Riya")
e2 = Employee("Aman")
print(Employee.total())
print(Employee.is_valid_name(""))
```

OUTPUT

```
2
False
```

LINE-BY-LINE

`total()` uses `cls` to read the shared `count` without needing any specific object. `is_valid_name()` needs no object or class context at all — it's a plain helper function living inside the class.

REAL-WORLD ANALOGY

In a company: an **instance method** is 'this employee submits their timesheet'; a **class method** is 'HR reports total employee count'; a **static method** is 'a general company policy checker' unrelated to any specific employee record.

COMMON BEGINNER MISTAKE

Using a static method when you actually need to access class-level data (like `count`) — a static method literally cannot access `cls` or `self`.

INTERVIEW POINT

'Difference between @classmethod and @staticmethod?' is asked in almost every Python interview — **classmethod** can access/modify class state via `cls`; **staticmethod** cannot access either.

Aspect	Instance Method	Class Method	Static Method
First param	self	cls	none
Access instance data?	Yes	No	No
Access class data?	Yes (via self.__class__)	Yes	No
Decorator	None needed	@classmethod	@staticmethod
Typical use	Normal object behavior	Alternate constructors	Utility/helper logic

12. @property — Pythonic Getters & Setters

DEFINITION

The `@property` decorator lets a method be accessed like an attribute (no parentheses), commonly used to add validation/computation while keeping clean attribute-style syntax instead of explicit `get_`/`set_` calls.

WHY IT IS USED

It gives you the safety of getter/setter methods with the clean, natural syntax of plain attribute access — considered the idiomatic Python approach.

SYNTAX

```
@property
def x(self): return self._x
@x.setter
def x(self, val): self._x = val
```

EXAMPLE

```
class Account:
    def __init__(self, balance):
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    @balance.setter
    def balance(self, amount):
        if amount < 0:
            raise ValueError("Negative balance not allowed")
        self._balance = amount

a = Account(500)
print(a.balance)
a.balance = 700
print(a.balance)
```

OUTPUT

```
500
700
```

LINE-BY-LINE

`a.balance` looks like plain attribute access but actually calls the `@property` getter method; `a.balance = 700` calls the `@balance.setter` method, which validates before storing.

REAL-WORLD ANALOGY

Like a smart thermostat display — you just 'read the temperature' or 'set the temperature' through a simple dial, while validation logic runs invisibly behind the scenes.

COMMON BEGINNER MISTAKE

Naming the private backing attribute the same as the property (e.g. both called `balance`) causes infinite recursion inside the getter/setter — always back it with a differently-named attribute like `_balance`.

INTERVIEW POINT

'@property lets you change internal implementation later without breaking any external code that already uses dot-attribute syntax' — a strong answer to 'why use property over plain attributes?'

12. Association, Aggregation & Composition (HAS-A relationships)

DEFINITION

These describe how independent objects relate to each other (as opposed to inheritance's IS-A relationship). **Association** is the most general 'uses/knows about' link. **Aggregation** is a HAS-A relationship where the contained object can exist independently of the container. **Composition** is a stronger HAS-A relationship where the contained object's lifetime depends entirely on the container.

WHY IT IS USED

These model real-world 'ownership' or 'usage' relationships between objects that aren't parent-child, which is extremely common in real applications.

EXAMPLE

```
# Aggregation - Department exists even if Teacher object is removed
class Teacher:
    def __init__(self, name):
        self.name = name
class Department:
    def __init__(self, teacher):
        self.teacher = teacher   # just holds a reference

# Composition - Engine cannot exist meaningfully without Car
class Engine:
    def __init__(self):
        self.status = "running"
class Car:
    def __init__(self):
        self.engine = Engine()   # Car creates & owns its Engine

c = Car()
print(c.engine.status)
```

OUTPUT

```
running
```

LINE-BY-LINE

In aggregation, `Department` merely references an externally-created `Teacher` — the teacher can outlive the department. In composition, `Car` itself creates its `Engine` inside `__init__` — if the car object is destroyed, its engine is destroyed too.

REAL-WORLD ANALOGY

Association: a Student and a Library (they interact but neither owns the other). **Aggregation**: a Department and a Teacher (teacher can move to another department). **Composition**: a House and its Rooms (a room can't exist without that specific house).

COMMON BEGINNER MISTAKE

Treating aggregation and composition as identical — the key test interviewers use is: 'can the contained object logically exist independently of the container?' Yes → aggregation. No → composition.

INTERVIEW POINT

'Difference between aggregation and composition?' → Both are HAS-A; composition implies ownership and shared lifetime, aggregation implies a looser, independent reference.

Relationship	Type	Example	Lifetime dependency
Inheritance	IS-A	Dog IS-A Animal	N/A (structural)
Association	USES-A	Student uses a Library	Fully independent
Aggregation	HAS-A (weak)	Department HAS-A Teacher	Independent
Composition	HAS-A (strong)	Car HAS-A Engine	Dependent

12. is vs ==, isinstance() & issubclass()

DEFINITION

== checks **value equality** (calls __eq__). `is` checks **identity** — whether two variables point to the exact same object in memory. `isinstance(obj, Class)` checks if an object belongs to a class (or its subclasses). `issubclass(Class1, Class2)` checks if one class inherits from another.

WHY IT IS USED

These are used constantly for correct comparisons and type-safety checks, especially before calling type-specific logic on an unknown object.

SYNTAX

```
a == b       # value equality
a is b       # identity
isinstance(obj, Cls)
issubclass(Sub, Base)
```

EXAMPLE

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
print(a == b)
print(a is b)
print(a is c)

class Animal: pass
class Dog(Animal): pass
d = Dog()
print(isinstance(d, Animal))
print(issubclass(Dog, Animal))
```

OUTPUT

```
True
False
True
True
True
```

LINE-BY-LINE

`a == b` is `True` because their contents match, but `a is b` is `False` because they're two separate list objects in memory. `c = a` makes c point to the same object as a, so `a is c` is `True`. `d` is recognized as an `Animal` via inheritance.

REAL-WORLD ANALOGY

== is like comparing two people's exam scores (same value, different people). `is` is like asking 'are you looking at the exact same answer sheet' (same physical object).

COMMON BEGINNER MISTAKE

Using `is` to compare values (e.g. `if x is 5`) — this can work by accident for small cached integers but is unreliable and considered bad practice; always use == for value comparison.

INTERVIEW POINT

A classic trap question: 'Why does `a is b` sometimes return True for small integers/strings but False for lists?' → CPython caches/interns small ints and some strings, but never interns mutable objects like lists.

SECTION 13

Dunder / Magic Methods

The hooks that make your objects behave like built-ins

13. What Are Dunder Methods?

DEFINITION

"Dunder" = **D**ouble **UNDER**score. These are special methods with names like `__init__` that Python calls automatically in response to built-in syntax or functions (creating an object, printing it, checking its length, comparing it, etc.) — never called directly by name in normal use.

```
class Book:
    def __init__(self, title, pages):
        self.title = title
        self.pages = pages

    def __str__(self):
        return f"Book: {self.title}"          # for print() / str()

    def __repr__(self):
        return f"Book({self.title!r}, {self.pages})"  # for debugging/repr()

    def __len__(self):
        return self.pages                      # for len()

    def __add__(self, other):
        return self.pages + other.pages        # for +

    def __eq__(self, other):
        return self.pages == other.pages       # for ==

b1 = Book("Python 101", 300)
b2 = Book("DSA Guide", 300)
print(b1)
print(repr(b1))
print(len(b1))
print(b1 + b2)
print(b1 == b2)
```

OUTPUT

```
Book: Python 101
Book('Python 101', 300)
300
600
True
```

REAL-WORLD ANALOGY

Dunder methods are like standardized electrical sockets — any appliance (object) that fits the socket shape (implements the right dunder method) automatically works with the house's wiring (Python's built-in functions/operators).

INTERVIEW POINT

Know the difference between `__str__` (readable, for end users, used by `print()`) and `__repr__` (unambiguous, for developers/debugging, used in the shell and inside containers like lists).

Dunder Method	Triggered By	Purpose
__init__	ClassName(args)	Initialize a new object's attributes
__str__	print(obj), str(obj)	Human-readable string representation
__repr__	repr(obj), shell echo	Unambiguous, developer-facing representation
__len__	len(obj)	Define what 'length' means for the object
__add__	obj1 + obj2	Define behavior of the + operator
__eq__	obj1 == obj2	Define value-based equality

🔥 Interview Traps

The questions designed to catch people who memorized definitions but didn't understand the concept.

Is method overloading directly supported in Python?

No. Redefining a method with a different signature simply overwrites the previous definition. Python simulates overloading using default arguments or *args/**kwargs.

Is every child class automatically polymorphic?

No. Polymorphism requires the child to actually override a method used generically. A child that only adds new, unrelated methods isn't demonstrating polymorphism by itself.

Difference between inheritance and polymorphism?

Inheritance is about acquiring/reusing a parent's code (IS-A relationship). Polymorphism is about the same method call producing different behavior depending on the actual object type. Overriding (which needs inheritance) is what enables polymorphism.

Difference between == and is?

== checks value equality (calls __eq__). is checks identity — whether both names point to the exact same object in memory.

Difference between a class variable and an instance variable?

A class variable is shared across all objects (one copy on the class). An instance variable is unique per object (own copy in each object's __dict__).

Difference between @classmethod and @staticmethod?

@classmethod receives cls and can access/modify class-level state. @staticmethod receives neither self nor cls and cannot access instance or class state — it's just a grouped utility function.

Difference between aggregation and composition?

Both are HAS-A relationships. In aggregation the contained object can exist independently of the container. In composition, the contained object's lifetime is tied to the container — if the container is destroyed, so is the contained object.

Does Python enforce private variables with __var?

No. Double underscore only triggers name mangling (renaming to _ClassName__var internally) — it's still technically accessible, just discouraged from direct outside use.

Can you instantiate a class that inherits from ABC but hasn't implemented all abstractmethods?

No — Python raises a TypeError at instantiation time if any abstract method remains unimplemented.

What does super() actually depend on?

It depends on the class's MRO (Method Resolution Order), not simply 'the direct parent' — this matters heavily with multiple inheritance.

⚡ 30-Second Revision

Every major term, boiled down to one line. Read this page right before walking into the interview.

OOPS: Programming style organizing code as objects bundling data + behavior.

Class: Blueprint defining attributes and methods.

Object: A concrete instance created from a class.

__init__: Special method that initializes a new object.

self: Refers to the current object inside instance methods.

Encapsulation: Bundling data + methods; restricting direct access.

Inheritance: Child class reuses parent's attributes/methods (IS-A).

super(): Calls the parent class's method/constructor via MRO.

Method Overriding: Child redefines a parent method with the same signature.

Polymorphism: Same interface, different behavior at runtime.

Method Overloading: Not natively supported in Python; simulated via defaults/*args.

Operator Overloading: Redefining operators (+, ==...) for custom objects via dunders.

Duck Typing: Behavior matters more than declared type.

Abstraction: Hiding implementation, exposing only essentials (ABC + @abstractmethod).

Instance variable: Unique per object, set via self.

Class variable: Shared by all objects of the class.

@classmethod: Takes cls; operates on the class itself.

@staticmethod: Takes neither self nor cls; plain utility function.

@property: Lets a method be accessed like an attribute.

Association: General USES-A relationship between independent objects.

Aggregation: Weak HAS-A; contained object can exist independently.

Composition: Strong HAS-A; contained object's life depends on container.

is vs ==: is checks identity (same object); == checks value equality.

isinstance(): Checks if an object belongs to a class (or subclass).

issubclass(): Checks if one class inherits from another.

Dunder methods: Double-underscore hooks (__init__, __str__...) triggered by built-ins.

💻 Output Prediction

15 snippets. Cover the answer with your hand, predict the output first, then check yourself.

SNIPPET 1

```
class A:
    x = 10
a1 = A()
a2 = A()
a1.x = 20
print(a1.x, a2.x, A.x)
```

ANSWER

OUTPUT

```
20 10 10
```

a1.x = 20 creates a new instance attribute on a1 only; the shared class attribute A.x (and a2.x) remain untouched at 10.

SNIPPET 2

```
class Parent:
    def show(self):
        print("Parent")
class Child(Parent):
    def show(self):
        print("Child")

objs = [Parent(), Child()]
for o in objs:
    o.show()
```

ANSWER

OUTPUT

```
Parent
Child
```

Each object calls its own class's overridden show() — classic runtime polymorphism.

SNIPPET 3

```
class Demo:
    def __init__(self):
        self.__secret = 5

d = Demo()
print(d._Demo__secret)
```

ANSWER

OUTPUT

```
5
```

__secret is name-mangled to _Demo__secret, not truly private — it's still reachable with the mangled name.

SNIPPET 4

```
class Counter:
    count = 0
    def __init__(self):
        Counter.count += 1

c1 = Counter()
c2 = Counter()
c3 = Counter()
print(Counter.count)
```

ANSWER

OUTPUT

```
3
```

Every new object increments the shared class variable count, so all three instances contribute to one total.

SNIPPET 5

```
class A:
    def greet(self):
        print("Hi from A")
class B(A):
    def greet(self):
        super().greet()
        print("Hi from B")

B().greet()
```

ANSWER

OUTPUT

```
Hi from A
Hi from B
```

super().greet() runs the parent's version first, then B adds its own print — both execute in order.

SNIPPET 6

```
a = [1, 2]
b = [1, 2]
print(a == b)
print(a is b)
```

ANSWER

OUTPUT

```
True
False
```

== compares values (equal contents), while is compares identity (two distinct list objects in memory).

SNIPPET 7

```
class Shape:
    def area(self):
        return 0
class Square(Shape):
    def __init__(self, s):
        self.s = s
    def area(self):
        return self.s * self.s

shapes = [Shape(), Square(4)]
print([s.area() for s in shapes])
```

ANSWER

OUTPUT

```
[0, 16]
```

List comprehension calls area() polymorphically; Shape returns the default 0, Square returns its overridden calculation.

SNIPPET 8

```
class Employee:
    def __init__(self, name):
        self.name = name
    @staticmethod
    def info():
        print("Employees work at TechCorp")

Employee.info()
```

ANSWER

OUTPUT

```
Employees work at TechCorp
```

A static method can be called directly on the class without creating any object, since it needs no self or cls.

SNIPPET 9

```
class Point:
    def __init__(self, x):
        self.x = x
    def __eq__(self, other):
        return self.x == other.x

p1 = Point(5)
p2 = Point(5)
print(p1 == p2)
print(p1 is p2)
```

ANSWER

OUTPUT

```
True
False
```

__eq__ is overloaded to compare the x values (True), but p1 and p2 are still two separate objects, so is returns False.

SNIPPET 10

```
class Base:
    def __init__(self):
        print("Base init")
class Derived(Base):
    def __init__(self):
        print("Derived init")

Derived()
```

ANSWER

OUTPUT

```
Derived init
```

Since Derived overrides __init__ WITHOUT calling super().__init__(), Base's constructor never runs at all.

SNIPPET 11

```
class Animal:
    def sound(self):
        print("Some sound")
class Dog(Animal):
    pass

d = Dog()
d.sound()
print(isinstance(d, Animal))
```

ANSWER

OUTPUT

```
Some sound
True
```

Dog doesn't override sound(), so it directly inherits and uses Animal's version; isinstance() confirms the IS-A relationship.

SNIPPET 12

```
class Number:
    def __init__(self, val):
        self.val = val
    def __add__(self, other):
        return Number(self.val + other.val)
    def __str__(self):
        return str(self.val)

n1 = Number(3)
n2 = Number(4)
print(n1 + n2)
```

ANSWER

OUTPUT

```
7
```

n1 + n2 calls __add__, producing a new Number(7); printing it calls __str__, which returns the string 7.

SNIPPET 13

```
def greet(name, msg="Hello"):
    print(msg, name)

greet("Aman")
greet("Aman", "Welcome")
```

ANSWER

OUTPUT

```
Hello Aman
Welcome Aman
```

Since Python lacks true overloading, a default argument lets one function handle both a 1-arg and 2-arg call style.

SNIPPET 14

```
class A:
    def show(self): print("A")
class B:
    def show(self): print("B")
class C(A, B):
    pass

C().show()
```

ANSWER

OUTPUT

```
A
```

With multiple inheritance class C(A, B), Python's MRO checks A before B (left-to-right order in the class definition), so A.show() wins.

SNIPPET 15

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self): pass

try:
    s = Shape()
except TypeError as e:
    print("Error caught")
```

ANSWER

OUTPUT

```
Error caught
```

Shape has an unimplemented abstract method, so instantiating it directly raises TypeError, which is caught here.

🧠 Interview Questions

40+ questions across four difficulty levels — practice explaining each answer out loud, not just recognizing it.

Beginner 11 Qs

- Q What is Object-Oriented Programming, and why is it used?
- Q What is the difference between a class and an object?
- Q What is the purpose of the __init__() method?
- Q What is self used for in a class?
- Q What are the four pillars of OOPS?
- Q What is the difference between an instance variable and a class variable?
- Q How do you create multiple objects from the same class?
- Q What is a constructor in Python?
- Q What does public, protected, and private mean in Python's naming convention?
- Q What is inheritance? Give a real-world example.
- Q What is the difference between a method and a function in Python?

Intermediate 16 Qs

- Q What is encapsulation, and how is it implemented in Python?
- Q Explain the different types of inheritance in Python with examples.
- Q What is method overriding? How is it different from overloading?
- Q Does Python support method overloading? If not, how do we simulate it?
- Q What is super() used for, and how does it relate to MRO?
- Q Explain polymorphism with an example that isn't inheritance-based.
- Q What is operator overloading? Name three dunder methods used for it.
- Q What is duck typing, and how does it relate to Python being dynamically typed?
- Q What is abstraction, and how do you implement it using the abc module?
- Q What is the difference between @classmethod and @staticmethod?
- Q What is @property used for, and why is it preferred over get_/set_ methods?
- Q What is the difference between association, aggregation, and composition?
- Q What is the difference between is and ==?
- Q What do isinstance() and issubclass() do?
- Q What are dunder/magic methods? Name at least four and their triggers.
- Q What is name mangling, and why does Python use it for double-underscore attributes?

Advanced 10 Qs

- Q Explain Method Resolution Order (MRO) and how Python resolves the diamond problem.
- Q How would you implement a custom iterable class using dunder methods?
- Q What is the difference between __str__ and __repr__? When would output differ?
- Q How does Python's garbage collector interact with objects that override __del__?
- Q Can a class inherit from multiple abstract base classes? What are the constraints?
- Q How would you make a class's instances immutable in Python?
- Q Explain how @property interacts with inheritance — can a child class override a property?

Q What is a metaclass, and how does it relate to class creation?

Q How do abstract classes differ from Protocols (structural typing) introduced in typing?

Q What happens internally when you call super() with no arguments inside a method?

Scenario-Based 8 Qs

Q You need to design a Library system with Book, Member, and Librarian — how would you use inheritance vs composition here?

Q A payroll system needs different salary calculations for Manager, Developer, and Intern — how would polymorphism help?

Q How would you prevent a class's critical internal attribute from being modified directly by external code?

Q You need one function that can add two numbers or two custom Vector objects — how do you design this given Python has no method overloading?

Q Design a class hierarchy for Vehicle → Car, Bike, Truck where every vehicle must implement fuel_efficiency() but the base Vehicle should not be instantiable directly.

Q How would you decide between aggregation and composition when modeling a University and its Departments?

Q A teammate's code breaks because they compared two custom objects using == and got unexpected False — how do you fix and explain it?

Q How would you refactor a class with ten separate get_x()/set_x() methods into more Pythonic code?

TOTAL
45 interview questions across Beginner, Intermediate, Advanced, and Scenario-based categories.

🏆 Coding Practice

15 problems — Easy, Medium, and Interview-level. Attempt each one fully before turning to the solutions.

Problem 1: Book Class Easy

TASK

Create a Book class with title, author, and price attributes, plus a display() method that prints all three.

Hint: Use __init__ to set the three attributes, and a plain instance method for display().

Problem 2: Bank Account with Encapsulation Easy

TASK

Create an Account class with a private __balance. Add deposit() and withdraw() methods that never allow the balance to go negative.

Hint: Use a double-underscore attribute and validate the amount inside each method before changing __balance.

Problem 3: Class Variable Counter Easy

TASK

Create a Student class that tracks the total number of students created using a class variable, exposed through a class method.

Hint: Increment a class-level variable inside __init__, and add a @classmethod that returns it.

Problem 4: Shape Hierarchy Easy

TASK

Create a base class Shape with a default area() returning 0, and subclasses Circle and Rectangle that override area() correctly.

> **Hint:** Override area() in each subclass using the correct formula; call area() on a list of mixed shape objects to test polymorphism.

Problem 5: Static Utility Class Easy

TASK

Create a MathUtils class with static methods is_even(n) and is_prime(n) — no object should be required to use them.

> **Hint:** Use @staticmethod for both; is_prime should check divisibility up to sqrt(n).

Problem 6: Vector Addition (Operator Overloading) Medium

TASK

Create a Vector class with x and y, and overload + so that adding two Vectors returns a new Vector with summed components. Also implement __str__.

> **Hint:** Implement __add__ to return Vector(self.x+other.x, self.y+other.y); implement __str__ to format it nicely.

Problem 7: Employee Payroll (Inheritance + Overriding) Medium

TASK

Create an Employee base class with a calculate_salary() method, then Manager and Intern subclasses that override it with different formulas.

> **Hint:** Base class returns a generic value; each subclass overrides calculate_salary() using its own bonus/stipend logic.

Problem 8: Abstract Payment System Medium

TASK

Create an abstract class Payment with an abstract method pay(amount), and concrete subclasses CreditCardPayment and UpiPayment.

> **Hint:** Use ABC and @abstractmethod on the base class; each subclass must implement pay() or it can't be instantiated.

Problem 9: Property-Based Temperature Converter Medium

TASK

Create a Temperature class storing celsius internally, with a fahrenheit property that computes on read and validates on write (no value below absolute zero).

> **Hint:** Use @property for the getter and @fahrenheit.setter for the setter; convert using F = C*9/5+32 both ways.

Problem 10: Library Aggregation Medium

TASK

Create a Library class that holds a list of externally-created Book objects (aggregation) — removing the Library should not delete the Book objects.

> **Hint:** Pass Book objects into Library's constructor or an add_book() method instead of creating them inside Library.

Problem 11: Custom Equality & Sorting Medium

TASK

Create a Product class with name and price. Implement __eq__ (equal if prices match) and __lt__ (so a list of Products can be sorted by price using sorted()).

> **Hint:** Implement __eq__ and __lt__ comparing self.price to other.price, then test with sorted(product_list).

Problem 12: Multiple Inheritance MRO Puzzle Interview-level

TASK

Design three classes A, B, C(A, B) where both A and B define a greet() method. Predict and verify which greet() actually runs, and explain using __mro__.

> **Hint:** Print C.__mro__ to see the exact resolution order Python computed, then confirm it matches which greet() executed.

Problem 13: Custom Iterable Class Interview-level

TASK

Create a class Range2 that behaves like a simplified range() by implementing __iter__ and __next__, usable directly inside a for-loop.

> **Hint:** __iter__ should return self; __next__ should raise StopIteration once the end value is reached.

Problem 14: Duck-Typed Notification System Interview-level

TASK

Create unrelated classes EmailNotifier, SmsNotifier, and PushNotifier that each implement a send(message) method, and write one function that works with any of them without checking their type.

> **Hint:** The function should simply call notifier.send(message) — no isinstance() checks required, demonstrating duck typing.

Problem 15: Composition-Based Car Builder Interview-level

TASK

Design a Car class composed of an Engine object and a GPS object that it creates internally, plus a start() method that reports status from both internal components.

> **Hint:** Car's __init__ should instantiate Engine() and GPS() itself (composition, not passed in), and start() should call methods on both.

✓ Coding Practice — Reference Solutions

One possible correct approach for each problem. Your own solution can look different and still be right.

Solution 1: Book Class

```
class Book:
    def __init__(self, title, author, price):
        self.title = title
        self.author = author
        self.price = price
    def display(self):
        print(self.title, self.author, self.price)
```

Solution 2: Bank Account with Encapsulation

```
class Account:
    def __init__(self, balance=0):
        self.__balance = balance
    def deposit(self, amt):
        self.__balance += amt
    def withdraw(self, amt):
        if amt > self.__balance:
            print("Insufficient funds")
        else:
            self.__balance -= amt
```

Solution 3: Class Variable Counter

```
class Student:
    count = 0
    def __init__(self, name):
        self.name = name
        Student.count += 1
    @classmethod
    def total(cls):
        return cls.count
```

Solution 4: Shape Hierarchy

```
class Shape:
    def area(self):
        return 0
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return 3.14 * self.r ** 2
class Rectangle(Shape):
    def __init__(self, l, w): self.l, self.w = l, w
    def area(self): return self.l * self.w
```

Solution 5: Static Utility Class

```
class MathUtils:
    @staticmethod
    def is_even(n): return n % 2 == 0
    @staticmethod
    def is_prime(n):
        if n < 2: return False
        for i in range(2, int(n**0.5)+1):
            if n % i == 0: return False
        return True
```

Solution 6: Vector Addition (Operator Overloading)

```
class Vector:
    def __init__(self, x, y):
        self.x, self.y = x, y
    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)
    def __str__(self):
        return f"({self.x}, {self.y})"
```

Solution 7: Employee Payroll (Inheritance + Overriding)

```
class Employee:
    def calculate_salary(self): return 0
class Manager(Employee):
    def calculate_salary(self): return 80000
class Intern(Employee):
    def calculate_salary(self): return 15000
```

Solution 8: Abstract Payment System

```
from abc import ABC, abstractmethod
class Payment(ABC):
    @abstractmethod
    def pay(self, amount): pass
class CreditCardPayment(Payment):
    def pay(self, amount): print(f"Paid {amount} via Credit Card")
class UpiPayment(Payment):
    def pay(self, amount): print(f"Paid {amount} via UPI")
```

Solution 9: Property-Based Temperature Converter

```
class Temperature:
    def __init__(self, celsius=0):
        self.celsius = celsius
    @property
    def fahrenheit(self):
        return self.celsius * 9/5 + 32
    @fahrenheit.setter
    def fahrenheit(self, f):
        if f < -459.67:
            raise ValueError("Below absolute zero")
        self.celsius = (f - 32) * 5/9
```

Solution 10: Library Aggregation

```
class Book:
    def __init__(self, title): self.title = title
class Library:
    def __init__(self):
        self.books = []
    def add_book(self, book):
        self.books.append(book)
```

Solution 11: Custom Equality & Sorting

```
class Product:
    def __init__(self, name, price):
        self.name, self.price = name, price
    def __eq__(self, other):
        return self.price == other.price
    def __lt__(self, other):
        return self.price < other.price
```

Solution 12: Multiple Inheritance MRO Puzzle

```
class A:
    def greet(self): print("A")
class B:
    def greet(self): print("B")
class C(A, B):
    pass
print(C.__mro__)
C().greet()  # prints "A" - A is checked before B
```

Solution 13: Custom Iterable Class

```
class Range2:
    def __init__(self, start, end):
        self.current, self.end = start, end
    def __iter__(self):
        return self
    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        val = self.current
        self.current += 1
        return val
```

Solution 14: Duck-Typed Notification System

```
class EmailNotifier:
    def send(self, msg): print("Email:", msg)
class SmsNotifier:
    def send(self, msg): print("SMS:", msg)
def notify(notifier, msg):
    notifier.send(msg)  # works with any object that has send()
```

Solution 15: Composition-Based Car Builder

```
class Engine:
    def status(self): return "Engine running"
class GPS:
    def status(self): return "GPS locked"
class Car:
    def __init__(self):
        self.engine = Engine()
        self.gps = GPS()
    def start(self):
        print(self.engine.status())
        print(self.gps.status())
```

Python OOPS — Last-Minute Interview Checklist

Run through this list 10 minutes before your interview. If you can explain each one out loud in under 30 seconds, you're ready.

- [ ] Class & Object
- [ ] Constructor & self

- [ ] Encapsulation
- [ ] Inheritance
- [ ] super()
- [ ] Method Overriding
- [ ] Polymorphism
- [ ] Operator Overloading
- [ ] Duck Typing
- [ ] Abstraction
- [ ] Class/Static methods
- [ ] Property
- [ ] Composition/Aggregation
- [ ] Dunder methods
- [ ] isinstance / issubclass
- [ ] is vs ==
